VEL SOFTWARE SOLUTIONS · EMPLOYEE TRAINING GUIDE · V1.0 · 3 NOV 2026

Software Delivery Handbook

From zero to 100% delivery - the stage-by-stage process of a professional software company, explained for beginners.

For	Details
Reader	New developers, testers, designers and project coordinators (basic knowledge is enough)
Covers	How a software company works · folder system · 14 stages with exact steps · templates · glossary
Example project	SiteDiary (sample offline site-report app: Flutter + NestJS + PostgreSQL)
Prepared by	**V.LOKESHWARAN** · VEL Software Solutions · velsoftwaresolutions.in@gmail.com
Companion document	Mobile App Company Operating System (management: costing, pricing, gates)

> **How to read**
> Start with Part A and B. Then follow Stage 0, 1, 2 ... in order. Each stage ends with a hand-in list and a manager sign-off.

★ Start here

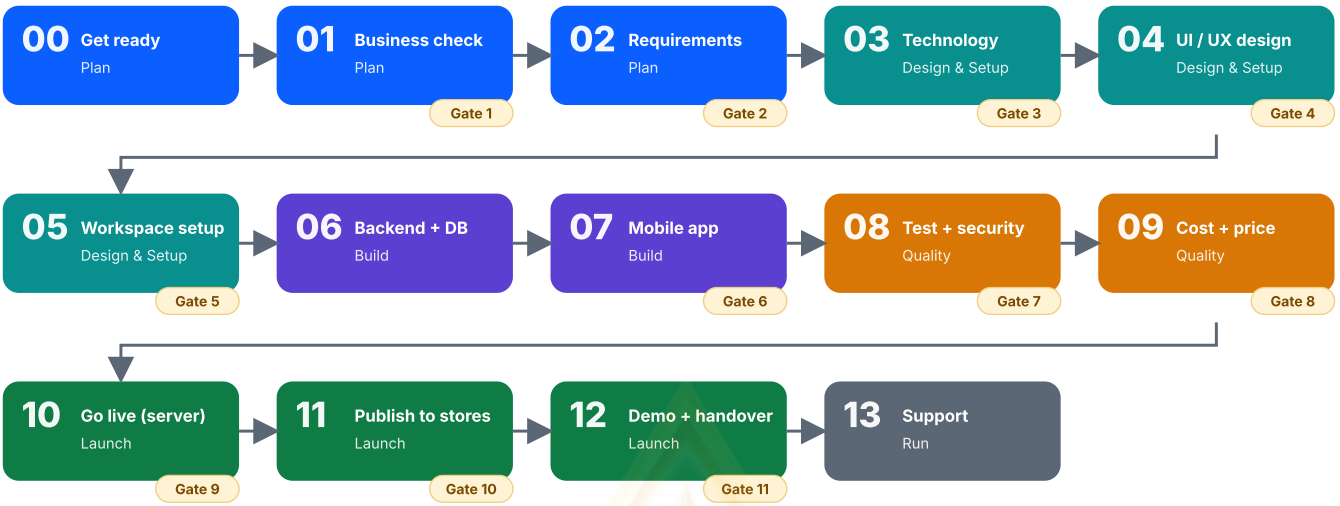
This handbook takes a person who knows only the basics from **an idea (0%) to a finished app delivered to a customer (100%)**, in the same way a professional software company does it. Every stage has the same layout so you always know what to expect:

Block	What it tells you
Stage at a glance	Time, people, tools, who approves
In plain words	Simple meaning of difficult terms
Before you start	What must already exist
Steps 1, 2, 3 …	Exactly what to do, in order, with commands and examples
What you must hand in	The files/proofs this stage produces
Quality check	Tick-list you must pass
Common mistakes	What beginners usually get wrong

How to use it

1. Read **Part A** once, fully. It explains how a software company works, so everything else makes sense.
2. Keep the **process map** open. It shows where you are: Stage 0 to Stage 13.
3. Work on **one stage at a time**. Do the steps in order. Do not skip a step because it looks small.
4. At the end of each stage, fill the **hand-in list**, tick the **quality check**, and ask your manager to sign the **gate form**.
5. If anything is unclear, use the **stuck ladder** in Part A (try 30 minutes → ask a teammate → ask the lead). Never guess silently.

> **The sample project**
> All examples use **SiteDiary**, a sample app that lets a site supervisor record daily work, attendance and photos, even without internet. Replace the names with your own project when you work.



#	Stage	Question it answers	You deliver	Approval
0	Get ready	Accounts, laptop, folders, rules	Working setup + first pull request	-
1	Business check	Is it worth building?	Market report	Gate 1
2	Requirements	What exactly will we build?	Frozen PRD + screen sheets	Gate 2
3	Technology	Tools and blueprint	Decision record + diagrams	Gate 3
4	UI / UX design	How it looks and works	Prototype + test results	Gate 4
5	Workspace setup	Repo, folders, checks	Ready repository	Gate 5
6	Backend + database	The server and its data	Tested API	-
7	Mobile app	The screens	Working app	Gate 6
8	Test + security	Find problems early	QA report	Gate 7
9	Cost + price	What it costs, what to charge	Price sheet	Gate 8
10	Go live	Server, domain, backups	Live system	Gate 9
11	Publish	Play Store + App Store	Released app	Gate 10
12	Demo + handover	Deliver to customer	Signed handover	Gate 11
13	Support	Keep it running	Monthly reports	-

> **Golden rule**
> Do not start a stage until the previous gate is signed. If a stage looks too big, break it into tickets of at most one day each.

A How a software company works

A.1 How a software company turns an idea into a product

Customer or idea → Product Manager writes requirements → Designer designs screens → Developers build → QA tests → DevOps deploys → Release to stores → Customer uses it → Support + updates

IN PLAIN WORDS: SOFTWARE COMPANY

It is a factory for software. Instead of machines it has **process**: the same steps for every project, so quality does not depend on luck.

A.2 Who is who

Team / role	What they do in simple words	You ask them when...
Founder / Managing Director	Decides what the company builds and signs the big approvals	A gate needs final approval, or money is involved
CTO / Tech Lead	Chooses technology, protects code quality, reviews difficult code	You are unsure how to build something
Product Manager (PM)	Talks to customers, writes requirements, controls scope, schedule and budget	You are unsure *what* to build or something changes
UI/UX Designer	Designs how the app looks and how people use it	A screen is unclear or does not fit
Mobile Developer	Builds the app people install on phones	App screens, device problems
Backend Developer	Builds the server and APIs the app talks to	Data, login, server errors
Database / DevOps Engineer	Keeps databases, servers, deployments and backups running	Deployments, environments, secrets
QA Engineer (tester)	Tries to break the app before customers do; decides if it can be released	Test cases, bug severity
Sales / Support	Demos, contracts, customer questions after delivery	Customer complaints and requests

In a small team one person may wear several hats. The jobs still exist - ask which hat someone is wearing when they give you an instruction.

A.3 How work moves (tickets)

IN PLAIN WORDS: TICKET

A small written task with a number, an owner, a deadline and a clear 'done' definition, stored in the project tracker. All work starts from a ticket.

Ticket status	Meaning	Who moves it
Backlog	Idea or task not planned yet	PM
To do	Planned for this sprint, ready to start	PM
In progress	You are working on it right now (max 2 at a time)	You
In review	Code finished; another person is checking it	You → Reviewer
In QA	Merged to the test server; tester verifies it	QA
Done	Meets the Definition of Done and tester approved	QA / PM
Blocked	You cannot continue; you wrote why and who can unblock	You

A.4 The daily loop of every developer

1 Pick a task → 2 New branch → 3 Write code + test → 4 Run checks → 5 Pull request → 6 Review → 7 Merge → 8 Update tracker

A.5 Meetings and routines

Meeting / routine	When	Length	What to say or bring
Daily stand-up	Every morning	15 min	Yesterday I did..., today I will..., I am blocked by...
Sprint planning	Start of 2-week sprint	1 h	Ask questions until you understand every task
Demo	End of sprint	30 min	Show working software, not slides
Retrospective	End of sprint	30 min	What went well, what to improve (be honest, be kind)
Weekly report	Friday	10 min to write	Template in Appendix
1-to-1 with lead	Weekly	20 min	Progress, learning goals, problems

A.6 Four environments

IN PLAIN WORDS: ENVIRONMENT

A separate copy of the system for a separate purpose, so experiments never touch real customers.

Environment	Where it runs	Who uses it	Data	Rule
Local	Your laptop	You	Fake	Break things freely here
Dev	Company test server	Developers	Fake	Updates automatically when code is merged
Staging	Copy of production	QA, PM, customer demo	Fake / anonymised	Last stop before customers
Production	Live server	Real customers	**Real**	Only approved releases; never experiment here

A.7 Company rules you must follow from day one

Rule	Why
Never put passwords, keys or customer data in code, chat or screenshots	Use the secret manager; keys leak through Git history forever
Use two-factor authentication on email, GitHub and cloud accounts	Stolen password alone must not be enough
Work only on company-approved tools and storage	Client code on a personal drive is a legal and security problem
One task = one branch = one pull request	Small changes are easy to review and to undo
Nobody merges their own code without a review	A second pair of eyes catches mistakes
Never test with real customer data	Use synthetic (fake) data
Write it down	If it is not in the ticket, document or message, it did not happen

A.8 When you are stuck - the stuck ladder

1. Try for **30 minutes**: read the error message completely, search it, check the docs.
2. Write the problem in 3 lines: what you wanted, what happened, what you tried.
3. Ask a **teammate** with that note.
4. Still stuck after another 30 minutes: ask the **lead**. Mark the ticket *Blocked*.
5. Production problem or customer data risk: tell the lead **immediately**, do not wait.

> **Professional habits that get noticed**
> Update tickets before asked · reply to messages within working hours · say 'I do not know yet, I will find out by 3 pm' instead of staying silent · estimate honestly · admit mistakes early.

A.9 What a Gate is

IN PLAIN WORDS: GATE

A checkpoint between two stages. A manager checks your evidence and signs. No signature = you do not start the next stage. This protects the customer's money and your time.

GATE APPROVAL FORM (COPY FOR EVERY GATE)

Project / Stage / Gate	PRJ-2026-001 · Stage __ · Gate __
Prepared by / date	_____
Evidence links	1) ____ 2) ____ 3) ____
Checklist passed?	☐ yes ☐ no (list failures)
Decision	☐ Approved ☐ Revise ☐ Rejected
Approver name / signature / date	_____
Comments	_____

B Folder system and naming

B.1 Folder system on the company drive

IN PLAIN WORDS: WHY FOLDERS MATTER

A new colleague must find any document in under a minute. Same structure for every project = no searching, no confusion, easy audits.

```
VEL-Software/                          <- shared company drive (backed up)
  00-Company/
    Policies/  Templates/  Brand/  Contracts/  Finance/(restricted)
  01-Projects/
    PRJ-2026-001-sitediary/            <- one folder per project; ID + short name
      01-Research/         market report, interview notes, evidence register
      02-Requirements/     PRD, screen specs, change requests
      03-Architecture/     tech decision (ADR), diagrams, threat model
      04-Design/           Figma links, exports, design tokens, usability report
      05-Source/           <- Git repository lives here (clone) - code is NOT stored in
the drive copy
      06-QA/               test plan, test cases, test reports, security checklist
      07-Costing-Pricing/  cost sheet, price sheet, price-source screenshots
      08-Release/          store listings, screenshots, release notes, checklists
      09-Delivery/         manuals, training, handover form, customer sign-off
      10-Support/          tickets, incident reports, maintenance log
      99-Archive/          old versions, closed items
```

Create it yourself (Linux / macOS / Git Bash on Windows):

```
$ mkdir -p ~/VEL-Software/{00-Company/{Policies,Templates,Brand,Contracts},01-Projects}
```

```
$ cd ~/VEL-Software/01-Projects
```

```
$ mkdir -p PRJ-2026-001-sitediary/{01-Research,02-Requirements,03-Architecture,04-
Design,05-Source,06-QA,07-Costing-Pricing,08-Release,09-Delivery,10-Support,99-Archive}
```

```
$ ls PRJ-2026-001-sitediary
```

B.2 Naming rules

Item	Convention	Example
Project folder	PRJ-YYYY-NNN-shortname	PRJ-2026-001-sitediary
Document	`PRJ-ID_DocType_vMAJOR.MINOR_YYYY-MM-DD.ext`	PRJ-2026-001_PRD_v1.0_2026-11-20.pdf
Git branch	type/ticket-short-description	feature/SD-14-attendance-form

Item	Convention	Example
Git commit	type(scope): message	feat(S-014): add attendance form
Release tag	vMAJOR.MINOR.PATCH	v1.0.0
Ticket ID	PROJECTCODE-number	SD-14
Bug ID	BUG-number	BUG-231

DO THIS

Use lowercase and hyphens in folders, no spaces. Put the **version and date** in every document name. Never name files *final_final2.docx*.

B.3 Folder system inside the code repository

```
sitediary/                          <- the Git repository (what developers share)
  README.md                 <- what this project is + how to run it (read first!)
  CONTRIBUTING.md           <- team rules: branches, commits, reviews
  .gitignore                <- list of files Git must never store (secrets, build output)
  .env.example              <- NAMES of settings (no real passwords)
  .github/workflows/        <- automatic checks that run on every pull request
  mobile/                   <- the phone app (Flutter)
    lib/
      core/                 <- shared helpers: errors, constants, utilities
      config/               <- environment settings (dev / staging / prod URLs)
      services/             <- talks to outside things: API, storage, notifications
      models/               <- data shapes (Site, Report, Worker)
      repositories/         <- one place to read/write each kind of data (API or local DB)
      features/             <- one folder per feature: auth/, sites/, reports/, sync/
      widgets/              <- reusable UI pieces (buttons, cards)
      navigation/           <- screens and routes
      state/                <- app state management
    assets/                 <- images, icons, fonts
    test/                   <- automatic tests for logic and widgets
    integration_test/       <- tests that click through whole flows
  backend/                  <- the server (NestJS)
    src/                    <- server code, one folder per module (auth, sites, reports,
files)
    tests/                  <- server tests
    migrations/             <- database change scripts, numbered, never edited after merge
  admin-dashboard/          <- web admin (only if the requirements need it)
  docs/                     <- project knowledge that travels with the code
    requirements/ architecture/ api/ database/ ui-ux/ deployment/ testing/
  infra/                    <- server and deployment configuration files
```

IN PLAIN WORDS: REPOSITORY (REPO)

The project's code folder with its full history, stored on GitHub. Everyone gets a copy (clone), makes changes on a branch, and merges them back through a pull request.

NEVER DO THIS

Do not store source code only on your laptop or in a zip file by email. If it is not in Git, it does not exist.

VEL SOFTWARE SOLUTIONS
V.LOKESHWARAN
VEL
SOFTWARE SOLUTIONS

PLAN · STAGE 0 OF 13

00 Get ready

Accounts, laptop, folders and rules - before any project work

TIME GUIDE
1 - 2 days

YOU WORK WITH
Your lead / mentor

TOOLS
Laptop, email, GitHub, tracker, password manager

APPROVED BY
Lead

IN PLAIN WORDS: STAGE 0
Nothing is built here. You are only making sure you can work safely and others can find your work.

Steps

1 Get your accounts

Company email · GitHub (join the organisation) · project tracker · design tool · cloud console (view access only at first) · password manager. Turn on **two-factor authentication** on every one.

2 Prepare your laptop

Install: Git, VS Code, Node.js (LTS), Flutter SDK, Android Studio. macOS only for iOS: Xcode. Check each with the commands below.

```
$ git --version
```

```
$ node -v
```

```
$ flutter --version
```

```
$ flutter doctor -v
```

TIP
`flutter doctor` shows a × next to anything missing. Fix them one by one using its instructions.

3 Set your Git identity

```
$ git config --global user.name "Your Name"
```

```
$ git config --global user.email "you@company.com"
```

```
$ git config --global init.defaultBranch main
```

4 **Create your SSH key and add it to GitHub**

This is your secure ID card for GitHub.

```
$ ssh-keygen -t ed25519 -C "you@company.com"
```

```
$ cat ~/.ssh/id_ed25519.pub
```

Copy the printed line into *GitHub → Settings → SSH and GPG keys → New SSH key*. Never share the file without `.pub`.

5 **Create your folder system**

Use the commands in section B.1. Put the project folder in the company drive.

6 **Read the project documents**

README.md, CONTRIBUTING.md and the requirements (PRD) of the project you join. Write 5 questions you still have and ask them in the first 1-to-1.

7 **Do the 'hello' test**

Clone the repo, run the app on an emulator or phone, change one text, make a branch, commit and open a pull request. This proves your setup works.

```
$ git clone git@github.com:<org>/<repo>.git
```

```
$ cd <repo>/mobile
```

```
$ flutter pub get
```

```
$ flutter run
```

WHAT YOU MUST HAND IN (OUTPUT)

Screenshot of `flutter doctor` with no × for your targets · your first merged pull request · folder structure created · accounts list ticked

Quality check

- ☐ 2FA on all accounts

- ☐ flutter doctor clean for chosen targets
- ☐ SSH key works (`ssh -T git@github.com`)
- ☐ Folder structure created exactly as B.1
- ☐ I know who my lead and mentor are

Common mistakes

- Skipping 2FA 'for now'
- Installing tools from random websites - use official downloads only
- Saving code in Downloads or Desktop

VEL SOFTWARE SOLUTIONS V.LOKESHWARAN VEL SOFTWARE SOLUTIONS

01

PLAN · STAGE 1 OF 13

Business check

Is this worth building? Find out before spending money

TIME GUIDE
1 - 2 weeks

YOU WORK WITH
PM, Founder, 10+ real people

TOOLS
Google Play, App Store, Google Trends, Reddit, G2/Capterra, spreadsheet

APPROVED BY
Founder (Gate 1)

IN PLAIN WORDS: WHY RESEARCH FIRST

The most expensive mistake is building something nobody pays for. A week of research can save months of work.

Steps

1 Write the idea in one sentence

PROBLEM STATEMENT

Who has the problem?	Site supervisors in small construction firms
What is the problem?	Daily site records are lost or late because of poor network and paper
How do they solve it today?	Paper notebooks, WhatsApp photos
What would success look like?	Owner receives a complete report every evening

2 Find existing apps

Search Google Play and the App Store with 5 different keywords. Open the 5 best apps. Write: name, price, rating, number of reviews, last update date, main features.

3 Read the complaints

Open each app's 1-star and 2-star reviews. Tag each complaint: *missing feature* · *price* · *bug* · *support*. Count them. The most common tag is your opportunity.

4 Talk to 10 real customers

1. Ask for 30 minutes. Do not sell. Ask how they work today.

2. Questions: What was the last time this problem happened? What did it cost? What do you use now? What would you pay?
3. Write notes the same day. Ask for a 60-day pilot or a small deposit at the end.

5 Write every fact in the Evidence Register

ID	Fact	Source URL	Date accessed	Confidence
E-001	Competitor X charges ₹___/month	https://...	2026-11-05	High (official page)

NEVER DO THIS

Never write a number without a source. If you cannot find it, write **Unknown - research needed**.

6 Estimate market size bottom-up

```
TAM = number of possible customers × yearly revenue per customer
SAM = the part you can reach in your language/region/channel
SOM = the part you can realistically win in 3 years
```

Use sourced counts (e.g. government or industry numbers). Show your arithmetic.

7 Decide how it makes money

Model	Use when
Subscription	Customers get value every month (recommended default)
One-time paid app	Single-use tool
Freemium	Free users cost almost nothing to serve
Enterprise contract	Large buyers, custom needs

8 Score the idea and recommend

Use the scorecard in the OS document (§3): ≥70 Go · 50-69 Pivot · <50 Stop. Write one page: recommendation and why.

9 Present and get Gate 1

Send the report to the Founder. Gate 1 is approved only with evidence and interview proof.

WHAT YOU MUST HAND IN (OUTPUT)

Market Research Report (PDF) · Evidence Register (sheet) · interview notes · go/pivot/stop scorecard

Quality check

- ☐ Every number has URL + date + confidence
- ☐ ≥ 10 interviews done and noted
- ☐ ≥ 5 competitors compared
- ☐ Bottom-up sizing shows its formula
- ☐ Recommendation is clear

Common mistakes

- Asking friends who are not customers
- Copying statistics from random blogs
- Falling in love with the idea and ignoring the score

VEL SOFTWARE SOLUTIONS V.LOKESHWARAN

02 Requirements

PLAN · STAGE 2 OF 13

Write exactly what will be built - and freeze it

TIME GUIDE
1 - 2 weeks

YOU WORK WITH
PM, designer, tech lead, QA

TOOLS
Document editor, tracker, spreadsheet

APPROVED BY
Founder + customer (Gate 2)

IN PLAIN WORDS: SCOPE FREEZE

Agreeing in writing what is included. It protects the customer from surprise bills and the team from endless changes.

Steps

1 List users and roles

Who uses the app? (Owner, Manager, Supervisor, Worker). What may each role do? Draw a table: capability × role (see OS §4).

2 List the modules (big features)

Example: Login · Sites · Daily report · Photos · Offline sync · Notifications · Admin. Mark each **Must / Should / Could / Won't** (MoSCoW). The first release is only the Musts.

3 Draw the screen list

Give every screen an ID: S-001 Login, S-010 Site list, S-014 Attendance … This ID is used in tickets, tests and code.

4 Fill one Screen Sheet per screen

SCREEN SHEET - S-014 ATTENDANCE

Purpose / role	Record who worked today / Supervisor
Inputs	worker (pick list, required) · status (present/absent/half) · hours (0-16, step 0.5) · note (≤200 characters)
Validation	hours required if present; no duplicate worker per day
Buttons	Save → store on phone + send later · Submit → lock the day
Calls API	POST /v1/reports/{id}/attendance
Database	UPSERT attendance
Errors	E-4101 'Hours must be between 0 and 16'
Loading / offline	Skeleton rows · works fully offline · 'Pending sync' badge

5 Write what happens when things fail

Situation	App must...
No internet	Keep working; save locally; show 'Offline'
Slow internet	Show saved data first; retry; never lose typed data
Server error	Friendly message + Retry
Session expired	Save draft, ask login, restore draft
Wrong password	Generic message; lock after 5 tries
Phone restarted	Reopen on same screen with draft

6 Review with everyone

Designer checks screens, tech lead checks feasibility, QA checks that every rule can be tested. Fix all open questions.

7 Freeze the scope

Tag the document `PRD v1.0 - FROZEN`. From now on any new idea needs a **Change Request**.

CHANGE REQUEST

CR number / date	CR-001 / ____
Requested by	____
What and why	____
Hours and cost (use rate card)	____
Delay (days)	____
Decision	☐ Approve ☐ Reject ☐ Later - signed ____

WHAT YOU MUST HAND IN (OUTPUT)

PRD v1.0 (frozen) · all Screen Sheets · screen→API→database table · failure behaviour table · signed Gate 2

Quality check

- ☐ Every screen has an ID and a sheet
- ☐ Every input has type, validation, required/optional
- ☐ Offline rules written per feature
- ☐ Roles and permissions table complete
- ☐ Customer or Founder signed

Common mistakes

- 'The developer will figure it out' - write it
- Adding features during build without a change request
- Forgetting error and empty screens

DESIGN & SETUP · STAGE 3 OF 13

03 Technology & architecture

Choose the tools and draw the blueprint before building

TIME GUIDE
3 - 5 days

YOU WORK WITH
Tech lead, PM

TOOLS
Diagram tool, document editor

APPROVED BY
CTO (Gate 3)

IN PLAIN WORDS: ARCHITECTURE

The blueprint of the system: what parts exist, how they connect, and how data flows - like drawings before constructing a building.

Steps

1 Re-read the frozen requirements

List what really drives the technology: offline needed? photos? payments? how many users? which countries?

2 Ask: do we even need a mobile app?

If a responsive website or PWA can do the job, recommend it. Mobile apps cost more to build, publish and maintain.

3 Compare the options with the scoring table

Score each option 1-5 per criterion, multiply by weights, add up (example in OS §5). Highest score wins *unless* the team cannot support it.

IN PLAIN WORDS: FLUTTER / NATIVE

Native = separate Android (Kotlin) and iOS (Swift) apps - best device access, double cost. **Flutter** = one codebase for both - faster and cheaper, most features available.

4 Draw the system diagram

```
  [ Phone app (Flutter) ] --HTTPS--> [ API server (NestJS) ] --> [ PostgreSQL database ]
     |   local SQLite DB            |        |  \--> [ File storage: photos, PDFs ]
     |   (offline data + outbox)    |        \----> [ Push service: FCM ]
     \--> works offline, syncs later  \--> [ Logs + monitoring ]
  [ Admin web dashboard ] --HTTPS--> same API server
```

Each box is one thing you must build, host or buy. Each arrow is a connection that must be secure.

5 Design the data (ER diagram)

List the things the system remembers (Organisation, User, Site, Worker, Report, Attendance, Photo) and how they connect (a Site has many Reports; a Report has many Attendance rows). Draw boxes and lines.

6 List the API endpoints

Method + path	Purpose
POST /v1/auth/otp/verify	Log in
GET /v1/sites	List sites
POST /v1/reports/{id}/attendance	Save attendance
POST /v1/uploads	Upload photo
POST /v1/reports/{id}/submit	Submit day

IN PLAIN WORDS: API

A menu of requests the server understands. The phone asks, the server answers with data.

7 Decide login and offline rules

Login method (OTP/password), how long sessions last, which data may be changed offline, and what happens when two devices change the same thing (conflict rule). Write each rule in one sentence.

8 Do a basic security think-through

- ☐ What data is sensitive?
- ☐ Where is it stored on the phone? (secure storage for tokens)

- ☐ Who can see whose data? (every request is checked on the server)
- ☐ What if the phone is lost?
- ☐ Which outside SDKs will we add, and what do they collect?

9 Write the Decision Record and get Gate 3

ADR-001 - FRAMEWORK DECISION

Context	Offline required, 6-person team, Android + iOS
Options	Flutter, Native, React Native, PWA
Decision	Flutter + NestJS + PostgreSQL
Because	Score table, team skills, cost
Risks / revisit when	Heavy device features needed later
Approved by / date	CTO ___

WHAT YOU MUST HAND IN (OUTPUT)

Decision Record (ADR) · system diagram · ER diagram · API endpoint list · login/offline/security notes · Gate 3

Quality check

- ☐ Options compared with scores
- ☐ Web/PWA alternative considered
- ☐ Diagram shows every component and connection
- ☐ Data model reviewed by backend and mobile developers
- ☐ Security questions answered

Common mistakes

- Choosing a tool because it is popular
- No offline rule, then surprise duplicate data
- Skipping the diagram and 'keeping it in head'

DESIGN & SETUP · STAGE 4 OF 13

04 UI / UX design

Decide how the app looks and feels - and test it with real users

TIME GUIDE
2 - 4 weeks

YOU WORK WITH
Designer, PM, 5 test users

TOOLS
Figma (or similar), prototype link

APPROVED BY
PM + customer (Gate 4)

IN PLAIN WORDS: UI VS UX

UI = how it looks (colours, buttons). **UX** = how easy it is to use (steps, clarity).

Steps

1 Map the user journeys

For each role draw the path from opening the app to finishing the main task. Mark where things can go wrong.

2 Draw the screen list and navigation

Use the same screen IDs as the requirements. Decide bottom tabs, menus and back behaviour.

3 Sketch low-fidelity wireframes

Black-and-white boxes only. Focus on **what is on the screen and where**, not colours. Review with the PM.

4 Create the design system (tokens)

Token	Example
Colours	Primary #0B5FFF, Success #0F7B45, Error #B3261E, Text #17202A
Type scale	Title 22, Body 16, Caption 12
Spacing	4 / 8 / 16 / 24 px
Radius	8 px cards, 24 px buttons

TIP

Define once, reuse everywhere. Never type a colour by hand in a screen.

5 Design the high-fidelity screens

Use Material guidance for Android and Human Interface guidance for iOS. Touch targets at least 48 dp (Android) / 44 pt (iOS). Text contrast at least 4.5:1.

6 Design every state

State	Show
Loading	Skeleton or spinner
Empty	Explain + one action button
Error	What happened + Retry
Offline	Banner + 'pending' badge
Permission	Why camera/location is needed before the system popup

7 Make a clickable prototype

Link the screens for the 5 main journeys. A non-designer should be able to click through without help.

8 Test with 5 real users

Give a task ('Record today's attendance'). Watch silently. Note where they hesitate. Calculate task success rate. Fix major problems.

9 Hand off to developers

Share the Figma link, exported icons, tokens, and notes. Walk the developers through the prototype in a 30-minute meeting.

WHAT YOU MUST HAND IN (OUTPUT)

Figma file link · clickable prototype · design tokens · component list · usability test notes · Gate 4

Quality check

- ☐ Every PRD screen is designed in every state
- ☐ Accessibility checked (contrast, size, labels)
- ☐ 5 users tested; ≥80% task success
- ☐ Original design - nothing copied from competitors
- ☐ Developers acknowledged the hand-off

Common mistakes

- Designing only the happy path
- Tiny text and buttons
- Hand-off by chat message instead of a walkthrough

VEL SOFTWARE SOLUTIONS V.LOKESHWARAN VEL SOFTWARE SOLUTIONS

DESIGN & SETUP · STAGE 5 OF 13

05 Workspace setup

Create the repository, folders, checks and environments

TIME GUIDE
1 - 2 days

YOU WORK WITH
DevOps / tech lead

TOOLS
GitHub, Git, terminal, Flutter, Node

APPROVED BY
Tech lead (Gate 5)

IN PLAIN WORDS: BRANCH

A private copy of the code where you work on one task. When finished you propose it back through a **pull request**.

Steps

1 Create the repository

GitHub → Organisation → **New repository** → name `sitediary` → **Private** → add README. Never make client code public.

2 Clone it

```
$ cd ~/VEL-Software/01-Projects/PRJ-2026-001-sitediary
```

```
$ git clone git@github.com:<org>/sitediary.git 05-Source
```

```
$ cd 05-Source
```

3 Create the mobile and backend projects

```
$ flutter create --org in.velsoftwaresolutions --project-name sitediary mobile
```

```
$ npm install -g @nestjs/cli
```

```
$ nest new backend
```

TIP

`--org` becomes your Android/iOS app ID. Use a domain the company owns, reversed. It cannot be changed after publishing.

4 Create the standard folders

```
$ cd mobile/lib && mkdir -p core config services models repositories features
widgets navigation state && cd ../..
```

```
$ mkdir -p admin-dashboard docs/{requirements,architecture,api,database,ui-
ux,deployment,testing} infra .github/workflows
```

5 Add .gitignore and .env.example

```
# .gitignore (key lines)
.env
*.jks
*.keystore
key.properties
node_modules/
build/
.dart_tool/
coverage/
*.p12
*.mobileprovision
```

```
# .env.example  (names only - real values live in the secret manager)
DATABASE_URL=
JWT_SIGNING_KEY=
FCM_SERVICE_ACCOUNT=
EMAIL_API_KEY=
STORAGE_BUCKET=
```

NEVER DO THIS

Never commit a real password, key, keystore or customer file. Git remembers forever, even after deleting.

6 **Protect the main branch**

GitHub → Settings → Branches (or Rulesets) → add rule for `main`: require pull request · require 1 approval · require status checks · block force-push.

7 **Add automatic checks (CI)**

Create `.github/workflows/ci.yml` (sample in the OS document §7). It runs analysis, tests and build on every pull request. Red = do not merge.

8 **Write README and CONTRIBUTING**

README: what it is, how to run mobile and backend (exact commands). CONTRIBUTING: branch names, commit style, review rules.

9 **Create three environments**

dev, staging, production - each with its own database, API URL and keys. Ask the DevOps engineer; record URLs in `docs/deployment` (no secrets).

10 **First commit and the onboarding test**

```
$ git checkout -b chore/SD-1-project-setup
```

```
$ git add .
```

```
$ git commit -m "chore: initial project structure"
```

```
$ git push -u origin chore/SD-1-project-setup
```

Open a pull request. A teammate must be able to clone fresh and run everything in under 30 minutes.

The 8 Git commands you will use every day

Command	What it does
git status	See what changed
git checkout -b feature/SD-14-attendance	Make a new branch for your task
git add .	Select all changes for the next save
git commit -m "feat(S-014): attendance form"	Save a snapshot with a message
git push -u origin	Upload your branch to GitHub

Command	What it does
git pull	Download teammates' changes
git log --oneline	See history

WHAT YOU MUST HAND IN (OUTPUT)

Private GitHub repo · folder structure · protected main · CI green · three environments listed · README/CONTRIBUTING · Gate 5

Quality check

- ☐ Fresh clone runs in < 30 min
- ☐ No secrets in repo (run the secret scan)
- ☐ main is protected
- ☐ CI runs on pull requests
- ☐ Environments are separate

Common mistakes

- Working directly on main
- Committing .env or keystore
- One huge commit with 'changes' as the message

06

BUILD · STAGE 6 OF 13

Backend & database

Build the server, the database and the API the app will talk to

TIME GUIDE
3 - 6 weeks (with Stage 7 in parallel)

YOU WORK WITH
Backend dev, DB engineer, tech lead

TOOLS
VS Code, Node, PostgreSQL, Postman/Bruno

APPROVED BY
Reviewed at Gate 6

IN PLAIN WORDS: BACKEND

The part users never see: it stores data, checks who may do what, and answers the phone app's requests.

Steps

1 Run PostgreSQL on your laptop

```
$ docker run --name pg-dev -e POSTGRES_PASSWORD=devpass -p 5432:5432 -d
postgres:16
```

TIP

This password is for your laptop only. Never reuse it anywhere else.

2 Write database migrations

One numbered SQL file per change. **Never edit a migration after it is merged** - add a new one.

```
-- migrations/0004_create_attendance.sql
CREATE TABLE attendance (
  id         UUID PRIMARY KEY,
  report_id  UUID NOT NULL REFERENCES reports(id),
  worker_id  UUID NOT NULL REFERENCES workers(id),
  status     TEXT NOT NULL CHECK (status IN ('present','absent','half')),
  hours      NUMERIC(3,1) CHECK (hours BETWEEN 0 AND 16),
  note       TEXT,
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (report_id, worker_id)
);
```

IN PLAIN WORDS: MIGRATION

A recorded step that changes the database structure, so every environment can be upgraded the same way.

3 Create a module per feature

```
$ cd backend
```

```
$ nest g resource reports
```

```
$ npm install class-validator class-transformer
```

Modules: auth, sites, workers, reports, files, notifications.

4 Write the endpoint with validation

```
// backend/src/reports/dto/create-attendance.dto.ts
import { IsIn, IsNumber, IsOptional, IsString, IsUUID, Max, MaxLength, Min }
from 'class-validator';
export class CreateAttendanceDto {
  @IsUUID() workerId: string;
  @IsIn(['present', 'absent', 'half']) status: string;
  @IsOptional() @IsNumber() @Min(0) @Max(16) hours?: number;
  @IsOptional() @IsString() @MaxLength(200) note?: string;
}
```

```
// backend/src/reports/reports.controller.ts
@Controller('v1/reports')
export class ReportsController {
  constructor(private readonly service: ReportsService) {}

  @Post(':id/attendance')
  addAttendance(@Param('id') id: string, @Body() dto: CreateAttendanceDto) {
    return this.service.addAttendance(id, dto);   // business logic lives in
the service
  }
}
// main.ts: app.useGlobalPipes(new ValidationPipe({ whitelist: true }));
```

Controller = receives request. DTO = checks the input. Service = business logic. Repository = database access. Keep them separate.

5 Return errors in one standard shape

```
{ "code": "E-4101", "message": "Hours must be between 0 and 16", "request_id": "..." }
```

NEVER DO THIS

Never show stack traces or database errors to the app.

6 Add login and permission checks

Every endpoint must check *who is calling* and *whether they may see this data* (for example only their organisation's sites). Test by trying another user's ID.

7 Support offline syncing

Accept an `Idempotency-Key` header so a retry from the phone never creates a duplicate. Provide `GET …?updated_after=` so the phone downloads only changes.

8 Write tests as you go

```
// backend/tests/attendance.e2e-spec.ts (idea)
it('rejects hours above 16', async () => {
  await request(app.getHttpServer())
    .post('/v1/reports/R1/attendance')
    .send({ workerId: W1, status: 'present', hours: 17 })
    .expect(400);
});
```

Test the good case, the bad input and the forbidden case for every endpoint.

9 Document the API

Generate OpenAPI (Swagger) and save it in `docs/api`. Mobile developers use it as the contract.

10 Log safely

Log request id, user id, route, time and error. Never log passwords, tokens or personal details.

11 Open pull requests, one per task

Use the pull request template: what changed, ticket, how tested, screenshots/logs.

WHAT YOU MUST HAND IN (OUTPUT)

Working API on dev server · migrations · OpenAPI document · automated tests passing · merged pull requests

Quality check

- ☐ Every endpoint validates input
- ☐ Every endpoint checks permission
- ☐ Error format is standard
- ☐ Migrations run from empty database to latest
- ☐ Tests cover success, invalid and forbidden cases
- ☐ No secrets or personal data in logs

Common mistakes

- Trusting the app to validate (the server must always validate)
- Editing old migrations
- Business rules inside controllers
- Forgetting duplicate protection on retries

07

BUILD · STAGE 7 OF 13

Mobile app development

Build the screens people will use - one ticket at a time

TIME GUIDE
4 - 8 weeks (in 2-week sprints)

YOU WORK WITH
Mobile devs, designer, backend dev, QA

TOOLS
VS Code / Android Studio, Flutter, emulator, real phone

APPROVED BY
CTO + PM (Gate 6)

IN PLAIN WORDS: FRONTEND / MOBILE

Everything the user sees and touches on the phone. It talks to the backend through the API.

Steps

1 Run the app on an emulator and a real phone

```
$ flutter devices
```

```
$ flutter run
```

Enable *Developer options* → *USB debugging* on an Android phone to test on a real device.

2 Take one ticket and make a branch

```
$ git checkout main && git pull
```

```
$ git checkout -b feature/SD-14-attendance-form
```

Read the Screen Sheet S-014 and the Figma screen. Ask questions before coding.

3 Build the screen from the design

Use the shared widgets and design tokens. Never copy-paste colours. Build the loading, empty, error and offline states too.

4 Add input validation exactly as the sheet says

```
// mobile/lib/features/reports/validators.dart
String? validateHours(String? v) {
  if (v == null || v.isEmpty) return 'Hours required';
  final h = double.tryParse(v);
  if (h == null || h < 0 || h > 16) return 'Hours must be between 0 and 16';
  return null;
}
```

Validation messages must match the Screen Sheet text.

5 Write a test for it

```
// mobile/test/validators_test.dart
import 'package:flutter_test/flutter_test.dart';
import 'package:sitediary/features/reports/validators.dart';

void main() {
  test('rejects hours above 16', () => expect(validateHours('17'),
isNotNull));
  test('accepts 8 hours', () => expect(validateHours('8'), isNull));
}
```

```
$ flutter test
```

The test fails first, then passes after the code is right.

6 Connect to the API through a repository

The screen never calls the internet directly. It asks a **repository**, which decides: local database or server. This makes offline possible.

7 Make it work offline

IN PLAIN WORDS: OUTBOX

A to-do list stored on the phone. Every save is written locally first, then sent when internet returns; a unique ID prevents duplicates.

Outbox column	Meaning
id	Unique ID created on the phone (UUID)
entity / operation	e.g. attendance / upsert
payload	The data to send
created_at	When the user saved it

Outbox column	Meaning
attempts	How many times sending failed
status	pending / sent / needs attention

8 **Handle every failure**

No internet, timeout, 401 (login again), 5xx (retry with waiting), permission denied. Never lose what the user typed.

9 **Run the checks before pushing**

```
$ flutter analyze
```

```
$ flutter test
```

Fix every warning. Format with `dart format .`

10 **Open a pull request and answer review comments**

Describe what and why, link the ticket, attach a screenshot/short video. Reply to each comment: fix it, or explain politely.

11 **Demo at the end of the sprint**

Show real working screens on a phone to the PM. Not slides.

Definition of Done (a ticket is finished only when all are ticked)

- ☐ Matches the Screen Sheet and the design
- ☐ Validation + error messages as written
- ☐ Works offline where required
- ☐ Unit/widget test added and passing
- ☐ flutter analyze has no warnings
- ☐ Reviewed and approved by someone else
- ☐ Tested on a real device
- ☐ Ticket updated and demo-able

WHAT YOU MUST HAND IN (OUTPUT)

All frozen screens working · tests passing · demo recording · merged pull requests · Gate 6 review sign-off

Common mistakes

- Building screens that are not in the requirements
- Calling the server directly from the screen
- Testing only on the emulator
- Huge pull requests nobody can review

VEL SOFTWARE SOLUTIONS
V.LOKESHWARAN
VEL
SOFTWARE SOLUTIONS

QUALITY · STAGE 8 OF 13

08 Testing & security

Try to break it before customers do

TIME GUIDE
2 - 4 weeks

YOU WORK WITH
QA engineer, developers, tech lead

TOOLS
Real phones, Postman/Bruno, test sheet, bug tracker

APPROVED BY
CTO (Gate 7)

IN PLAIN WORDS: QA

Quality Assurance: a person (or process) whose job is to find problems early, when they are cheap to fix.

Steps

1 Write test cases from the Screen Sheets

For each rule write: steps → expected result. Example: enter 17 hours → message 'Hours must be between 0 and 16'.

2 Run functional tests

Go through every screen and every rule. Mark Pass/Fail in the traceability sheet (requirement → test → result).

3 Test on many devices

Test on	Minimum
Android	3 devices: old, mid, new; one with low memory
iOS	Current and previous major version
Screens	Smallest and largest phone
Rotation / tablet	Only if the requirements say so

4 Test the network

Wi-Fi, mobile data, very slow, airplane mode, switching networks, killing the app during upload.

5 Measure performance

Measure	Target
App start	≤ 3 seconds on a mid-range phone
Screen load (cached)	≤ 1 second
API response (95%)	≤ 500 ms
Crash-free sessions	≥ 99.5%

6 Do the security checks

- ☐ Login: wrong password, expired session, lockout
- ☐ Tokens stored in secure storage, not plain files
- ☐ User A cannot read user B's data (try changing IDs in requests)
- ☐ App uses HTTPS only
- ☐ No passwords/tokens in logs or screenshots
- ☐ Permissions asked only when needed
- ☐ Deep links and web views checked
- ☐ Outside SDKs reviewed: what do they collect?
- ☐ Account and data deletion works

Reference standards: OWASP MASVS (what to achieve) and MASTG (how to test).

7 Report bugs properly

BUG REPORT

ID / title	BUG-231 · Attendance not saved after reconnect
Severity	Sev-1 crash/data loss · Sev-2 major · Sev-3 minor · Sev-4 cosmetic
Steps to reproduce	1) Turn on airplane mode 2) Save attendance 3) Turn it off
Expected / actual	Row appears on server / nothing appears
Device / OS / app version	Redmi Note 11 · Android 13 · 1.0.0 (12)
Attachments	Screen recording + log

8 Fix and re-test

Developer fixes in a new branch; tester repeats the exact steps; run the full regression suite before release.

9

Write the QA & Security report

Summary: tests run, pass/fail, open bugs by severity, device list, security results, QA recommendation. **Gate 7 requires zero open Sev-1 and Sev-2.**

WHAT YOU MUST HAND IN (OUTPUT)

Test cases + results · bug list · device list · performance numbers · security checklist · QA & Security report · Gate 7

Quality check

- ☐ Every requirement has at least one test
- ☐ 0 open Sev-1/Sev-2 bugs
- ☐ Tested on real devices and bad networks
- ☐ Security checklist complete
- ☐ Automated tests run in CI

Common mistakes

- Only testing the happy path
- Reporting bugs without steps
- Releasing with 'known' serious bugs

QUALITY · STAGE 9 OF 13

09 Cost & price

Know exactly what it costs, then price it so the company earns

TIME GUIDE
2 - 3 days

YOU WORK WITH
PM, Finance, Founder

TOOLS
Spreadsheet, official pricing pages and calculators

APPROVED BY
Founders (Gate 8)

IN PLAIN WORDS: MARGIN

The share of the selling price that is profit. 30% margin on ₹100 price = ₹30 profit, ₹70 cost. Price = cost ÷ (1 − margin).

Steps

1 Get the rate card from your PM

Role	Hourly rate card (loaded)	Where it comes from
Product Manager	₹938	PM + Finance
UI/UX Designer	₹729	
Flutter Developer	₹885	
Backend Developer	₹938	
QA Engineer	₹521	
DevOps / Database	₹938	
Security specialist (external)	₹2,500	Quote

These are hourly **cost** rates the company uses (they already include overheads). Never invent your own.

2 List every task with hours

Break the project into tasks (design, login, offline sync, testing ...). For each task estimate three numbers per role: **best (O)**, **most likely (M)**, **worst (P)**.

```
Expected hours E = (O + 4 × M + P) ÷ 6
```

3 Multiply hours by rates

```
Task cost = Σ (hours of each role × that role's hourly rate)
```

Count each hour once. If one person does two jobs, give the hours to the job actually done.

4 Add non-labour costs

Tools, test devices, domain, legal review, store accounts. Every price needs the **official page link and the date you checked**.

5 Add monthly running cost

Use the official cloud calculators with your expected users (low / medium / high). Do not guess; do not assume every user is active daily.

6 Calculate cost and price

Line	SiteDiary example	How to calculate
Labour (expected)	₹15,00,549	Σ hours × rate for each task and role (PERT: (O + 4M + P) ÷ 6)
Setup & third-party	₹2,08,160	tools, devices, domain, store accounts, legal
Company overhead (12%)	₹1,80,066	labour × 12%
Risk allowance	₹2,88,427	50% × (pessimistic − expected labour)
Total cost	₹21,77,202	sum of the four lines above
Selling price (30% margin)	**₹31,10,289**	Total cost ÷ (1 − 0.30)
Minimum allowed price (15% margin)	₹25,61,414	Total cost ÷ (1 − 0.15)

7 Check the margin

If the price is below the minimum, you may not quote it. Ask the Founder.

8 Hand in the worksheet for Gate 8

Attach screenshots of every price source with date.

WHAT YOU MUST HAND IN (OUTPUT)

Cost worksheet · price sheet · source screenshots (URL + date) · Gate 8 approval

Full models

The management document *Mobile App Company Operating System* (Phase 8-9) contains the full numbers: user-scale simulation, three infrastructure models, SaaS tiers, break-even and maintenance packages.

Quality check

- ☐ Every price has official source + date
- ☐ No hour counted twice
- ☐ Three scenarios (best/likely/worst) shown
- ☐ Price ≥ minimum margin price
- ☐ Founder signed

Common mistakes

- Counting only developer salaries
- Forgetting testing, deployment, documentation and training
- Assuming free tiers last forever
- Quoting before getting the Gate approved

LAUNCH · STAGE 10 OF 13

10 Go live: server & domain

Put the backend on the internet safely

TIME GUIDE
2 - 4 days

YOU WORK WITH
DevOps, backend dev, QA

TOOLS
Cloud console, domain registrar, terminal, monitoring tool

APPROVED BY
CTO (Gate 9)

IN PLAIN WORDS: DEPLOY

Copying the tested software to the live server and switching it on, using a repeatable script rather than manual steps.

Steps

1 Buy and secure the domain

Use an accredited registrar. Turn on registrar lock and 2FA. Domain belongs to the company account, never a personal one.

2 Create the production environment

Production server, production database, file storage - separate from dev/staging, with separate credentials.

3 Store secrets in the secret manager

Put DATABASE_URL, JWT_SIGNING_KEY, etc. in the cloud secret store. The app reads them at start-up. Rotate keys on schedule.

NEVER DO THIS

Do not put production secrets inside the mobile app. Anything shipped inside an app can be extracted.

4 Run database migrations

Run them through the deployment pipeline, not by hand. Take a backup first.

5 **Deploy the backend**

Use the approved pipeline: tag a release → pipeline builds → deploys to staging → QA checks → manual approval → production.

```
$ git tag v1.0.0
```

```
$ git push origin v1.0.0
```

6 **Connect DNS**

Add the records **exactly as the hosting provider shows them** (A/CNAME/TXT). Never guess values. Wait for propagation; confirm HTTPS works (padlock).

7 **Turn on monitoring and alerts**

Uptime check, error rate, slow responses, disk and database health. Alerts go to the on-call person's phone.

8 **Verify backups by restoring one**

A backup you never restored is only a hope. Restore into a test database and check the data.

9 **Practise rollback**

Redeploy the previous version on staging and confirm it works. Write the steps in `docs/deployment`.

10 **Run the production smoke test**

Register, login, create a report, upload a photo, receive a push notification - using a test account.

WHAT YOU MUST HAND IN (OUTPUT)

Live API URL with HTTPS · monitoring screenshots · backup restore proof · rollback test proof · smoke-test result · Gate 9

Quality check

- ☐ HTTPS works; HTTP redirects
- ☐ Secrets are in the secret manager only
- ☐ Backup restored successfully
- ☐ Alerts reach a real person
- ☐ Rollback tested
- ☐ Dev and production URLs are separate

Common mistakes

- Deploying by hand from a laptop
- No backup test
- Same database for staging and production

VEL SOFTWARE SOLUTIONS V.LOKESHWARAN VEL SOFTWARE SOLUTIONS

11

LAUNCH · STAGE 11 OF 13

Publish to stores

Get the app onto Google Play and the App Store

TIME GUIDE
1 - 3 weeks incl. review time (not guaranteed)

YOU WORK WITH
Release manager, QA, designer

TOOLS
Android Studio, Xcode (Mac), Play Console, App Store Connect

APPROVED BY
CTO + PM (Gate 10)

IN PLAIN WORDS: STORE REVIEW

Google and Apple check each app. They may ask for changes or reject it. Nobody can promise an approval date - plan extra time.

Accounts and fees

Account	Fee	Status (checked 3 Nov 2026)	Needs
Apple Developer Program	US$99 per year (local currency; price varies by region)	**Verified** on Apple's official page	Organisation: legal entity + D-U-N-S number; no trade names/DBAs
Apple Enterprise Program	US$299 per year	Verified - internal company apps only, not for public stores	Not for our public apps
Google Play Console	US$25 one-time	Medium-High: Google help page via search; confirm on signup screen	Government ID and card in legal name may be requested

Re-check before paying

Official pages change. Open the official page on the day, screenshot the fee, and file it in `07-Costing-Pricing`.

Android steps

1

Fix the app ID and version

Package name never changes after publishing. In `pubspec.yaml`: `version: 1.0.0+1` (name+build number). Every upload needs a higher build number.

2 **Create the upload key (once)**

```
$ keytool -genkey -v -keystore upload-keystore.jks -keyalg RSA -keysize 2048 -validity 10000 -alias upload
```

NEVER DO THIS

Store the .jks file and its passwords in the company vault and one offline backup. If lost, updates become very hard. Never commit it.

3 **Build the Android App Bundle**

```
$ flutter build appbundle --release
```

Output: `build/app/outputs/bundle/release/app-release.aab`

4 **Create the store listing**

Title, short and full description, icon, feature graphic, phone screenshots, privacy policy URL, category, contact email.

5 **Complete Data safety and content forms**

Declare what data the app and its SDKs collect. It must match reality - Google can reject or remove apps that do not.

6 **Upload to Internal testing, then wider testing**

Invite testers by email. Fix problems. Check current rules for your account type on the day (new personal accounts may have extra testing steps).

7 **Submit for review and release gradually**

Start with 10% → 50% → 100% while watching crashes.

iOS steps

1 **Register the app ID and create the app record**

Apple Developer → Identifiers; App Store Connect → New App (name, bundle ID, language, SKU).

2 **Set signing**

In Xcode enable *Automatically manage signing* with the company team, or use the team's certificate process.

3 **Build the archive and upload**

```
$ flutter build ipa --release
```

Upload with Xcode Organizer or Transporter.

4 **Fill App Store Connect**

Screenshots for required device sizes, description, keywords, support URL, **App Privacy** answers, age rating, **demo account and notes for the reviewer**.

5 **TestFlight beta**

Internal testers first, then external. Collect crash reports and feedback.

6 **Submit for App Review**

Respond to reviewer messages within 24 hours. Release in phases.

After release - verification

- ☐ Install from the store on a clean phone
- ☐ Register + login + reset password
- ☐ Main workflow end-to-end on production
- ☐ Push notification arrives
- ☐ Crash reporting receives events
- ☐ Privacy policy and support links open

WHAT YOU MUST HAND IN (OUTPUT)

Signed AAB and iOS build · store listings · privacy declarations · testing records · release notes · Gate 10 · post-release verification

Common mistakes

- Losing the signing key
- Privacy form that contradicts what SDKs really collect
- No demo login for the Apple reviewer
- Using a personal account instead of the company's

12

LAUNCH · STAGE 12 OF 13

Demo, handover & delivery

Show it, teach it, hand it over - delivery 100%

TIME GUIDE
3 - 7 days

YOU WORK WITH
PM, Sales, support, customer

TOOLS
Staging/demo build, slides, manuals

APPROVED BY
Founder + customer (Gate 11)

IN PLAIN WORDS: HANDOVER

The formal moment when the customer receives the finished product and the responsibilities change. It must be written and signed.

Steps

1 Prepare the demo

Use a demo account with fake data. Write a 10-minute script: problem → three main workflows → admin view → limits → price.

2 Present honestly

Show what works. State what is not included. Never promise features that are not in the signed scope.

3 Prepare the documents

- ☐ User manual with screenshots
- ☐ Admin manual
- ☐ Quick-start one-pager
- ☐ Training slides
- ☐ FAQ
- ☐ Support contacts and hours

4 Train the customer

One live session per role. Let them do the tasks while you watch. Record the session.

5 Hand over access

Transfer accounts, domains, store listings and credentials through the secure vault - as agreed in the contract. Remove your personal access afterwards.

6 Sign the handover

HANDOVER FORM

Project / version delivered	PRJ-2026-001 · v1.0.0
Deliverables handed over	☐ app ☐ source code (per contract) ☐ manuals ☐ credentials ☐ training
Known issues	___
Support package and start date	___
Customer name / signature / date	___
Company name / signature / date	___

7 Start the support period

Open the support channel and schedule a check-in after one week and one month.

WHAT YOU MUST HAND IN (OUTPUT)

Demo recording · manuals · training record · signed handover form · Gate 11

Common mistakes

- Handing over without documents
- Leaving your personal accounts as owners
- No signed acceptance

13

RUN · STAGE 13 OF 13

Support & maintenance

Keep it working and improving

TIME GUIDE
Continuous

YOU WORK WITH
Support, developers, DevOps

TOOLS
Crash reporter, monitoring, ticket tracker

APPROVED BY
Support lead

IN PLAIN WORDS: MAINTENANCE

Software needs care after launch: phones update, servers age, new bugs appear, customers ask for improvements.

Steps

1 Watch the dashboards daily

Crashes, errors, slow APIs, failed syncs, server health.

2 Handle tickets by severity

Priority	Example	Response goal (Professional plan)
P1	App down, data at risk	4 business hours
P2	Major feature broken	1 business day
P3	Minor problem	Next monthly release

3 Update regularly

Monthly: dependency and security updates. Every OS season: test on new Android/iOS. Yearly: renew accounts, domain, certificates.

4 Review backups

Monthly check; quarterly restore drill.

5 Write incident reports

INCIDENT REPORT

What happened / when	___
Impact	___
Cause	___
Fix	___
How we prevent it	___

6 Change requests

New features always go through the change request form with cost and schedule.

7 Renew and report

Send the customer a monthly report: uptime, crashes, tickets, hours used.

Boundary

Support has a fixed number of hours per month in the signed package. Extra work is billed. Never promise unlimited support.

C Appendix: templates, glossary, help

C.1 Pull request template

```
## What changed
## Ticket (SD-14)
## How I tested it (commands / screenshots)
## Risks or things the reviewer should check
## Checklist
- [ ] Tests added   - [ ] flutter analyze clean   - [ ] No secrets   - [ ] Docs updated
```

C.2 Daily stand-up note

DAILY STAND-UP NOTE

Yesterday	____
Today	____
Blocked by	____ (who can help?)

C.3 Weekly report

WEEKLY REPORT (FRIDAY)

Tasks planned / done / carried over	__ / __ / __
Hours estimated vs actual	__ vs __
Bugs opened / fixed	__ / __
Delays and reason	____
Risks / help needed	____
Learned this week	____

C.4 Release notes

RELEASE NOTES	
Version / date	v1.0.0 · ____
New	____
Improved	____
Fixed	____
Known issues	____

C.5 Glossary

Term	Meaning in plain words
API	Menu of requests a server understands
Backend / frontend	Server side / what users see
Branch	Your private working copy of the code
Commit	A saved snapshot with a message
Pull request (PR)	Asking the team to review and merge your branch
CI/CD	Robots that test and deploy automatically
Build	Turning code into an installable app
Release	A tested version given to users
Staging / production	Test copy / live system
Migration	A recorded database change
Token	A temporary digital pass proving who you are
Endpoint	One URL of the API, e.g. /v1/sites
JSON	Text format for data, e.g. {"hours": 8}
SDK	Ready-made code library from another company
Idempotent	Safe to repeat - same result, no duplicates
Sync	Making phone and server data match
MVP	Smallest version that is useful
Sprint	A 2-week work period
Severity	How bad a bug is (Sev-1 worst)
Keystore	File that proves you own the Android app - protect it
Bundle ID	Permanent unique ID of an app, e.g. in.company.app

Term	Meaning in plain words
TestFlight	Apple's tool for beta testers
Gate	Approval checkpoint between stages
Scope	What is included in the project
Margin	Profit as a share of price

C.6 Troubleshooting guide

Problem	What to try
`flutter doctor` shows × Android licenses	Run `flutter doctor --android-licenses` and accept all
`Permission denied (publickey)` on git	Your SSH key is not added to GitHub or not loaded: repeat Stage 0 step 4
Merge conflict	Open the file, keep the correct lines, remove the <<<< markers, `git add`, `git commit`. Ask for help if unsure.
Port 5432 already in use	Another PostgreSQL is running: stop it or use a different port
App works in emulator not on phone	Check USB debugging, internet, permissions, and the API URL (emulator uses 10.0.2.2 for the laptop's localhost)
Push rejected (protected branch)	You pushed to main. Create a branch and open a pull request
CI is red	Open the CI log, find the first error, run the same command locally, fix, push
I committed a secret	Tell the lead **now**. The secret must be rotated; deleting the commit is not enough

C.7 The '100% delivered' checklist

- ☐ Market research approved (Gate 1)
- ☐ Requirements frozen and signed (Gate 2)
- ☐ Architecture approved (Gate 3)
- ☐ Design and prototype approved (Gate 4)
- ☐ Repo and environments ready (Gate 5)
- ☐ All frozen screens built and demoed (Gate 6)
- ☐ QA & security passed, zero Sev-1/2 (Gate 7)
- ☐ Cost and price approved (Gate 8)
- ☐ Production live, backup and rollback proven (Gate 9)
- ☐ Store release approved and verified (Gate 10)
- ☐ Handover signed by the customer (Gate 11)
- ☐ Support package active

C.8 Official documents to bookmark

Topic	Official documentation
Flutter	https://docs.flutter.dev/
Android	https://developer.android.com/docs
Apple Developer	https://developer.apple.com/documentation/
App Store Connect	https://developer.apple.com/help/app-store-connect/
Google Play Console	https://support.google.com/googleplay/android-developer/
NestJS	https://docs.nestjs.com/
PostgreSQL	https://www.postgresql.org/docs/
Git	https://git-scm.com/doc
GitHub	https://docs.github.com/
OWASP MASVS / MASTG	https://mas.owasp.org/
Firebase	https://firebase.google.com/docs

VEL Software Solutions · Developed by V.LOKESHWARAN · velsoftwaresolutions.in@gmail.com · © 2026 · Internal training document. Costs shown are examples; use the current rate card and verified prices.